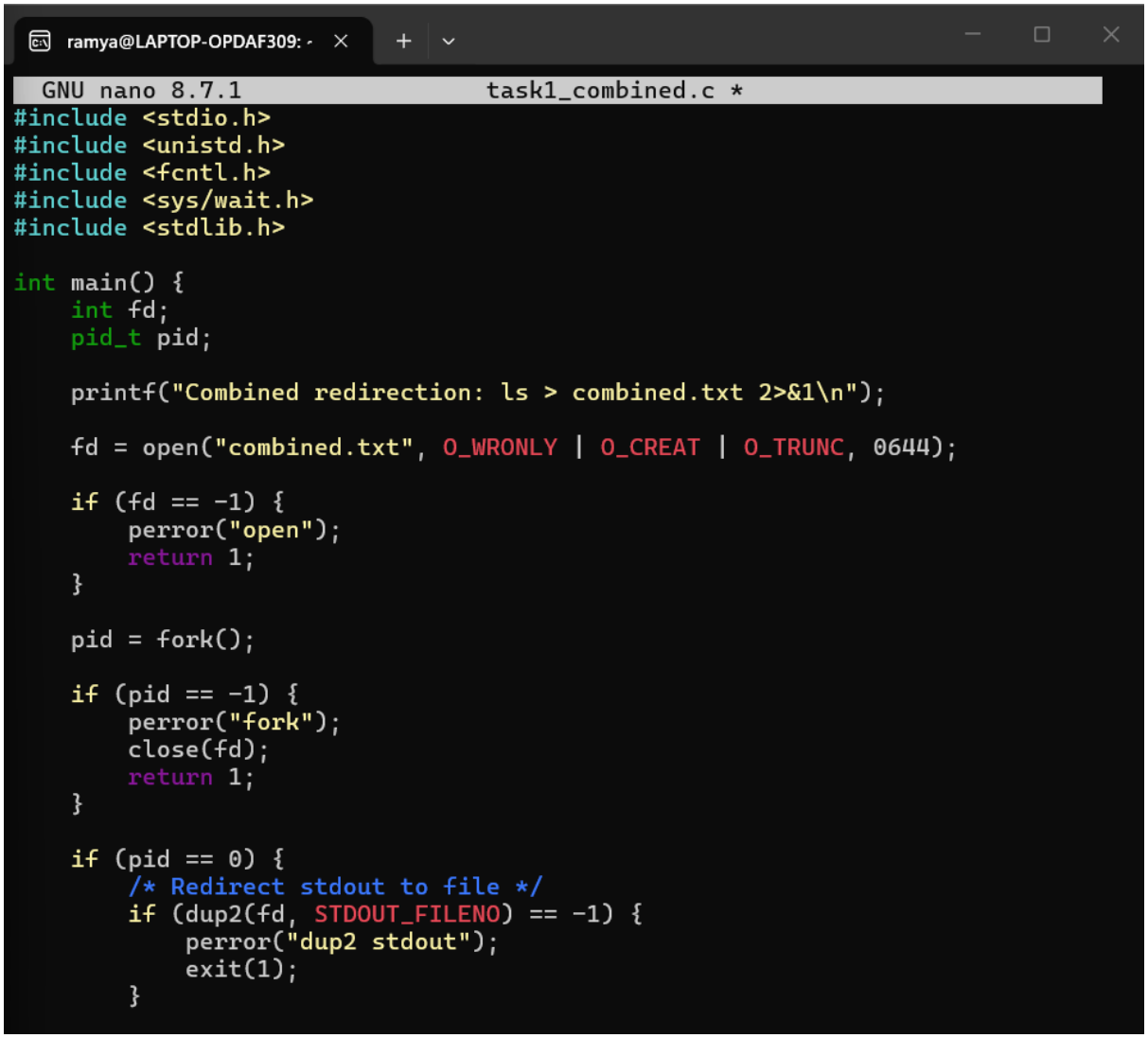
WEEK – 8

TASK 1 : TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs



```c
        /* Redirect stderr to the same stdout */
        if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1) {
            perror("dup2 stderr");
            exit(1);
        }

        close(fd);

        printf("This is standard output.\n");
        fflush(stdout);

        fprintf(stderr, "This is standard error.\n");
        fflush(stderr);

        exit(0);
    }

    close(fd);

    waitpid(pid, NULL, 0);

    printf("Combined redirection completed.\n");
    printf("Check combined.txt for both streams.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~$ mkdir skill8
ramya@LAPTOP-OPDAF309:~$ cd skill8
ramya@LAPTOP-OPDAF309:~/skill8$ nano task1_combined.c
ramya@LAPTOP-OPDAF309:~/skill8$ gcc task1_combined.c -o task1_combined
ramya@LAPTOP-OPDAF309:~/skill8$ ./task1_combined
Combined redirection: ls > combined.txt 2>&1
Combined redirection completed.
Check combined.txt for both streams.
ramya@LAPTOP-OPDAF309:~/skill8$ cat combined.txt
This is standard output.
This is standard error.
ramya@LAPTOP-OPDAF309:~/skill8$ cat combined.txt
This is standard output.
This is standard error.
ramya@LAPTOP-OPDAF309:~/skill8$ ls -l combined.txt
-rw-r--r-- 1 ramya ramya 49 Sep 28 03:50 combined.txt
ramya@LAPTOP-OPDAF309:~/skill8$
```

TASK 2 : To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

ramya@LAPTOP-OPDAF309:

GNU nano 8.7.1 task2_complex.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX 200

int main() {
    char input[MAX];
    char left[MAX];
    char right[MAX];
    char output[MAX];

    printf("Enter command:\n");
    printf("Example: cat input.txt | grep Linux > complex_output.txt\n");
    printf("> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *pipe_pos = strchr(input, '|');

    if (pipe_pos == NULL) {
        printf("Syntax error: pipe '|' is missing.\n");
        return 1;
    }

    char *redirect_pos = strchr(pipe_pos + 1, '>');

    if (redirect_pos == NULL) {
        printf("Syntax error: output redirection '>' is missing.\n");
        return 1;
    }
```

```c
    if (pipe_pos == input || redirect_pos == pipe_pos + 1) {
        printf("Syntax error: invalid command structure.\n");
        return 1;
    }

    *pipe_pos = '\0';
    strcpy(left, input);

    *redirect_pos = '\0';
    strcpy(right, pipe_pos + 1);

    strcpy(output, redirect_pos + 1);

    while (left[0] == ' ')
        memmove(left, left + 1, strlen(left));

    while (right[0] == ' ')
        memmove(right, right + 1, strlen(right));

    while (output[0] == ' ')
        memmove(output, output + 1, strlen(output));

    if (strlen(left) == 0 || strlen(right) == 0 || strlen(output) == 0) {
        printf("Syntax error: incomplete command.\n");
        return 1;
    }
```

```c
    printf("\n--- Execution Plan ---\n");
    printf("Command 1 : %s\n", left);
    printf("Pipe      : |\n");
    printf("Command 2 : %s\n", right);
    printf("Redirect  : > %s\n", output);

    int pipefd[2];

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 == -1) {
        perror("fork");
        return 1;
    }

    if (p1 == 0) {
        close(pipefd[0]);

        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("cat", "cat", "input.txt", NULL);

        perror("cat");
        exit(1);
    }
```

```
    pid_t p2 = fork();

    if (p2 == -1) {
        perror("fork");
        return 1;
    }

    if (p2 == 0) {
        close(pipefd[1]);

        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

        int fd = open(output, O_WRONLY | O_CREAT | O_TRUNC, 0644);

        if (fd == -1) {
            perror("output file");
            exit(1);
        }

        dup2(fd, STDOUT_FILENO);
        close(fd);

        execlp("grep", "grep", "Linux", NULL);

        perror("grep");
        exit(1);
    }
```

```
    close(pipefd[0]);
    close(pipefd[1]);

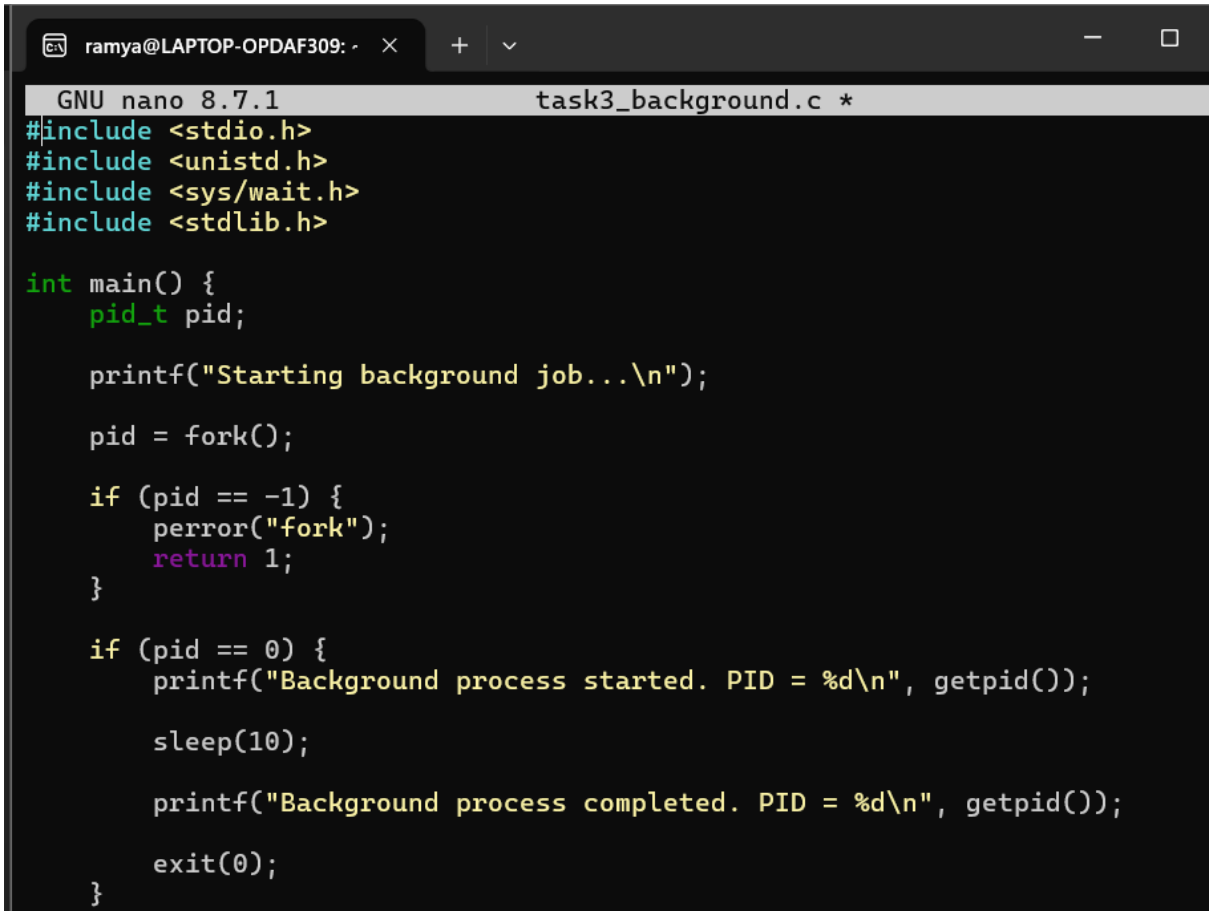
    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

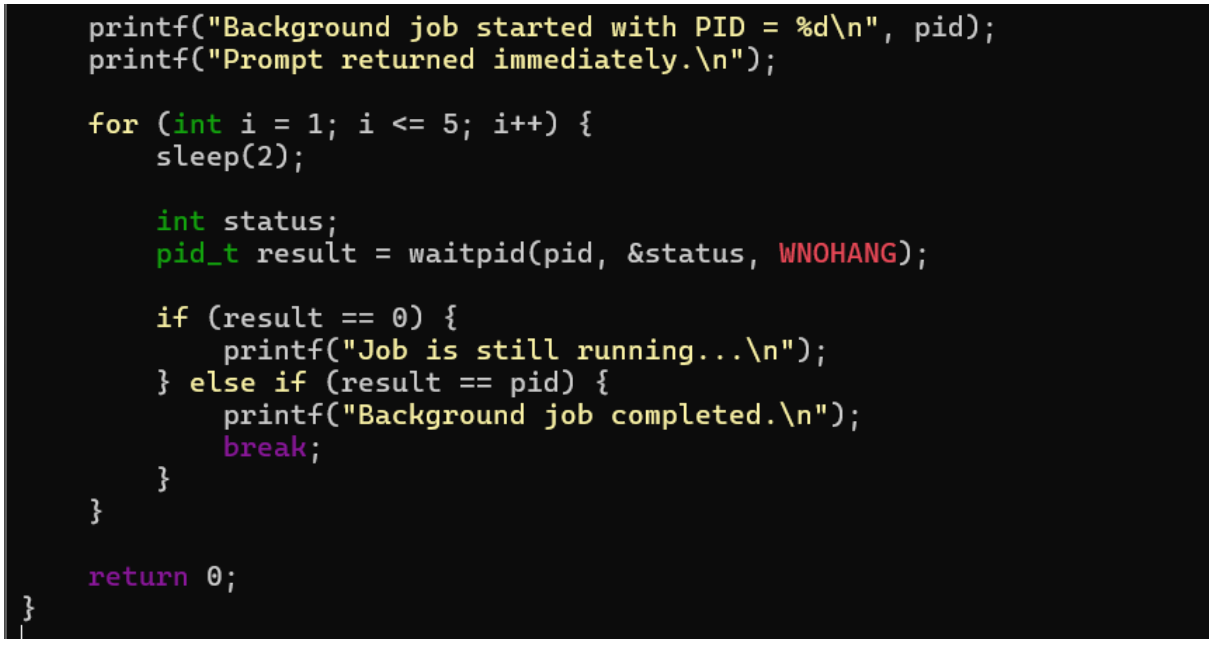
    printf("\nComplex command execution completed.\n");

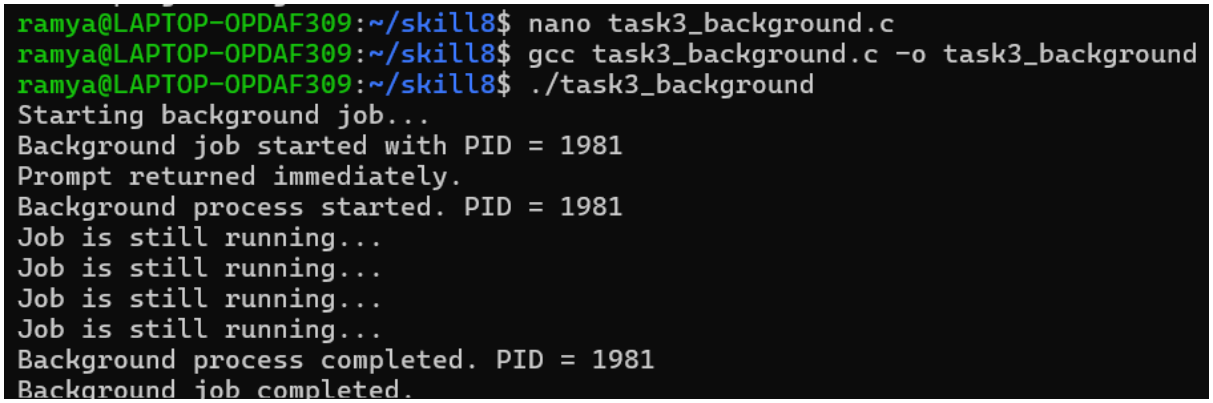
    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill8$ nano task2_complex.c
ramya@LAPTOP-OPDAF309:~/skill8$ echo "Hello Linux" > input.txt
ramya@LAPTOP-OPDAF309:~/skill8$ echo "Operating System" >> input.txt
ramya@LAPTOP-OPDAF309:~/skill8$ echo "Linux programming" >> input.txt
ramya@LAPTOP-OPDAF309:~/skill8$ cat input.txt
Hello Linux
Operating System
Linux programming
```

TASK 3 : To Detect Background Jobs, Launch Non-Blocking Processes, Retum Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks


ramya@LAPTOP-OPDAF309:
GNU nano 8.7.1 task3_background.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main() {
pid_t pid;
printf("Starting background job...\n");
pid = fork();
if (pid == -1) {
perror("fork");
return 1;
}
if (pid == 0) {
printf("Background process started. PID = %d\n", getpid());
sleep(10);
printf("Background process completed. PID = %d\n", getpid());
exit(0);
}


printf("Background job started with PID = %d\n", pid);
printf("Prompt returned immediately.\n");
for (int i = 1; i <= 5; i++) {
sleep(2);
int status;
pid_t result = waitpid(pid, &status, WNOHANG);
if (result == 0) {
printf("Job is still running...\n");
} else if (result == pid) {
printf("Background job completed.\n");
break;
}
}
return 0;
}



TASK 4 : To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

ramya@LAPTOP-OPDAF309:

GNU nano 8.7.1 task4_jobtable.c *

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

#define MAX_JOBS 3

typedef struct {
    int job_id;
    pid_t pid;
    int state;
} Job;

int main() {
    Job jobs[MAX_JOBS];
    int job_count = 0;

    printf("--- Creating Job Table ---\n");

    for (int i = 0; i < MAX_JOBS; i++) {
        pid_t pid = fork();

        if (pid == -1) {
            perror("fork");
            return 1;
        }

        if (pid == 0) {
            sleep(3 + i);
            exit(0);
        }

        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;
        jobs[job_count].state = 1;

        job_count++;
```

```
    }

    printf("\n--- Job Table ---\n");

    for (int i = 0; i < job_count; i++) {
        printf("Job ID: %d | PID: %d | State: Running\n",
               jobs[i].job_id, jobs[i].pid);
    }

    printf("\nMonitoring jobs...\n");

    while (job_count > 0) {

        for (int i = 0; i < job_count; i++) {
            int status;

            pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

            if (result == jobs[i].pid) {

                jobs[i].state = 0;

                printf("Job %d completed. PID = %d\n",
                       jobs[i].job_id, jobs[i].pid);

                for (int j = i; j < job_count - 1; j++) {
                    jobs[j] = jobs[j + 1];
                }

                job_count--;
                i--;
            }
        }
```

```
        if (job_count > 0) {
            printf("Active jobs: %d\n", job_count);
            sleep(1);
        }
    }

    printf("\nAll jobs completed.\n");
    printf("Job table is empty and consistent.\n");

    return 0;
}
```

```
ramya@LAPTOP-OPDAF309:~/skill8$ nano task4_jobtable.c
ramya@LAPTOP-OPDAF309:~/skill8$ gcc task4_jobtable.c -o task4_jobtable
ramya@LAPTOP-OPDAF309:~/skill8$ ./task4_jobtable
--- Creating Job Table ---

--- Job Table ---
Job ID: 1 | PID: 2038 | State: Running
Job ID: 2 | PID: 2039 | State: Running
Job ID: 3 | PID: 2040 | State: Running

Monitoring jobs...
Active jobs: 3
Active jobs: 3
Active jobs: 3
Job 1 completed. PID = 2038
Active jobs: 2
Job 2 completed. PID = 2039
Active jobs: 1
Job 3 completed. PID = 2040

All jobs completed.
Job table is empty and consistent.
```